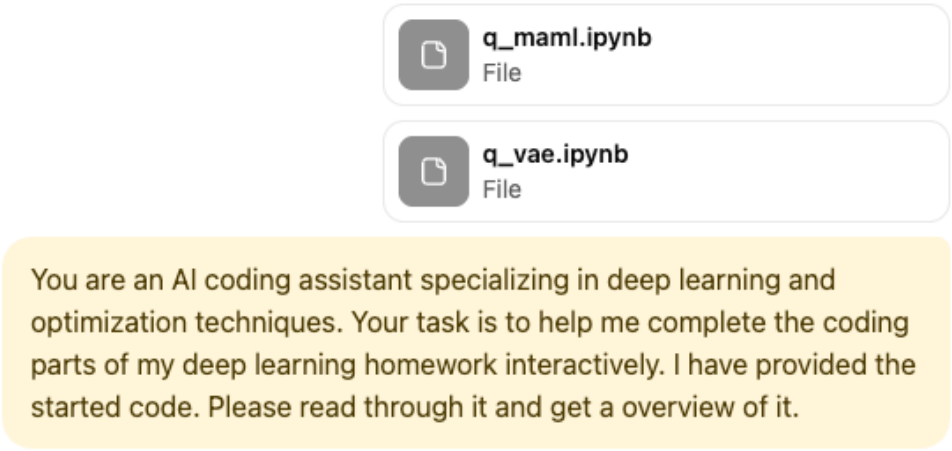
CS282 Special Participation B: GPT5-Pro on HW12

I use GPT5-Pro to solve HW12 written part in this special participation B.

For the MAML notebook, the missing pieces involved implementing the correct loss functions for binary classification tasks. Specifically, the regression-based MSE losses were replaced with the appropriate logistic loss in both the inner-loop task adaptation phase and the outer-loop meta-update phase. This ensured that each task's labels were transformed into $\{-1,+1\}$form and optimized using a log-likelihood–consistent objective, allowing the meta-learner to adapt effectively to classification tasks. Once these losses were added, the entire MAML workflow—sampling tasks, running inner gradient steps, and performing meta-updates—became fully functional.

For the VAE notebook, the missing implementations resided in the underlying codebase rather than the notebook itself. The first missing component was the reparameterization trick used to sample latent variables $z \sim q(z \mid x)$; this was completed by writing a sample_gaussian function that generates z, where ϵ is standard Gaussian noise. The second missing part was the full computation of the negative Evidence Lower Bound (ELBO), which combines the KL divergence between the approximate posterior and the prior with the negative log-likelihood of the data under the decoder' s Bernoulli distribution. Completing the negative_elbo_bound function provided the correct training objective required for VAE optimization and sampling. With these components implemented, the VAE model can now encode images, sample meaningful latent vectors, decode reconstructions, compute ELBO loss, and train end-to-end as intended.



Thought for 1m 37s ›

Here is a clean and clear summary of how to **fully complete both notebooks** (`q_maml.ipynb` and `q_vae.ipynb`), including the **exact code you should paste**.

✅ 1. Completing `q_maml.ipynb` (Classification Version of MAML)

There are **two TODOs** in this notebook:

1.1 TODO #1 — inside `meta_update_classification`

Replace the TODO block with the following logistic-loss implementation:

python Copy code

```python
# Convert continuous y into {+1, -1}
z_meta_t = my_sign(y_meta_t).float()

# Logistic loss:  sum_i log(1 + exp(-z_i * f_i))
loss = torch.sum(torch.log1p(torch.exp(- z_meta_t * y_meta_pred_t)))
```

Full function for copy-paste:

python Copy code

```python
def meta_update_classification(w_t, feature_weights_t, n_train_meta, phi_type, d, k_i
    x_meta = generate_x(n_train_meta, 'uniform_random')
    features_meta = featurize(x_meta, phi_type=phi_type, d=d, normalize=True)
    features_meta_t = torch.tensor(features_meta)

    y_meta = generate_y(features_meta, k_idx, k_val)
    y_meta = add_label_noise(y_meta, noise_prob)
    y_meta_t = torch.tensor(y_meta)

    y_meta_pred_t = features_meta_t @ w_t

    # ==== YOUR TODO (completed) ====
    z_meta_t = my_sign(y_meta_t).float()
    loss = torch.sum(torch.log1p(torch.exp(- z_meta_t * y_meta_pred_t)))
    # ==============================

    return loss
```

1.2 TODO #2 — inside the inner loop of `meta_learning_classification`

Replace the classification loss TODO with:

```python
z_t = my_sign(y_t).float()
loss = torch.mean(torch.log1p(torch.exp(- z_t * y_pred)))
```

Full block for copy-paste:

```python
with higher.innerloop_ctx(dm, opt_inner, copy_initial_weights=False, track_higher_gra

    for j in range(num_gd_steps):
        y_pred = mod(features_t)

        # ==== YOUR TODO (completed) ====
        z_t = my_sign(y_t).float()
        loss = torch.mean(torch.log1p(torch.exp(- z_t * y_pred)))
        # ================================

        opt_inner.zero_grad()
        loss.backward(retain_graph=True)
        opt.step(loss)
```

✅ 2. Completing `q_vae.ipynb`

The notebook itself is complete; **the TODOs are inside the codebase**:

- `codebase/utils.py` → `sample_gaussian`
- `codebase/models/vae.py` → `negative_elbo_bound`

Here is the correct implementation.

2.1 Implement `sample_gaussian` (Reparameterization Trick)

Paste this into `utils.py`:

python Copy code

```python
def sample_gaussian(mean, logvar):
    """
    Reparameterization trick:
        z = mean + std * eps, eps ~ N(0, I)
    """
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mean + std * eps
```

2.2 Implement `negative_elbo_bound` inside `vae.py`

This is the standard VAE ELBO:

$$\text{NELBO} = \text{KL}(q(z|x)\|p(z)) - \mathbb{E}_{q(z|x)}[\log p(x|z)]$$

2.2 Implement `negative_elbo_bound` inside `vae.py`

This is the standard VAE ELBO:

$$\text{NELBO} = \text{KL}(q(z|x)\|p(z)) - \mathbb{E}_{q(z|x)}[\log p(x|z)]$$

Paste this version (adapt names depending on your codebase):

python Copy code

```python
def negative_elbo_bound(self, x):
    # Encode to obtain q(z|x)
    mean, logvar = self.enc(x)

    # Sample z using reparameterization
    z = ut.sample_gaussian(mean, logvar)

    # Prior and posterior log-probabilities
    log_qz = ut.log_normal(z, mean, logvar)
    log_pz = ut.log_normal_standard(z)

    kl = torch.mean(log_qz - log_pz)

    # Decode; decoder output is logits for Bernoulli
    logits_x = self.dec(z)

    # Reconstruction term
    log_pxz = ut.log_bernoulli_with_logits(x, logits_x)
    rec = -torch.mean(log_pxz)

    nelbo = kl + rec

    stats = {
        "nelbo": nelbo.detach(),
        "kl": kl.detach(),
        "rec": rec.detach()
    }

    return nelbo, stats
```

⚠️ IMPORTANT

Make sure to check how your training loop calls this function:

- some expect `return nelbo, kl, rec`
- some expect `return nelbo, stats`